

Line-edit-pkg.lisp マニュアル

本プログラムは Common Lisp で記述した Common Lisp 用の readline ライブラリ相当の機能を備えたプログラムです。以下のプログラム群で構成されています。

```
line-edit-pkg.lisp      ;; 行編集機能を提供するパッケージ。
history-pkg.lisp       ;; 履歴機能を提供するパッケージ。
history-repl.lisp      ;; REPL 機能を提供するパッケージ。
package-util.lisp      ;; パッケージ関連機能を提供するパッケージ。
print-color-string.lisp ;; xterm256 端末操作と色出力機能を提供するパッケージ。
support-functions.lisp  ;; 各種サポート用関数を提供するパッケージ。
```

これらのファイルを任意のディレクトリ（以下、~/Lisp/line-edit と仮定）に置き、次のファイルを ~/.config/line-edit ディレクトリに置きます。

```
emacs-mode.lisp      ;; Emacs 互換編集コマンド用のキー・バインド定義ファイル。
vi-mode.lisp         ;; vi 互換編集コマンド用のキー・バインド定義ファイル。
vi-mode-function.lisp ;; Emacs 用関数と 1 対 1 対応していない編集用関数の定義ファイル。
WordMaster-mode.lisp ;; WordMaster 互換編集コマンド用のキー・バインド定義ファイル。
WordMaster-mode-function.lisp ;; Emacs 用関数と 1 対 1 対応していない編集用関数の定義ファイル。
set-editor-mode.lisp ;; 前回、使用していたエディタ・モードを記録しておくファイル。
cursor-info.lisp     ;; 端末がカーソル色変更機能を備えているかの記述用ファイル。
init-repl-prompt.lisp ;; 起動時のプロンプト設定記述用ファイル。
registered-message-file.lisp ;; 日本語に加え、英語、ドイツ語などのメッセージ・ファイル。
syntax-info-list.lisp ;; Common Lisp の約 800 種の関数などの補完定義ファイル。
user-info-list.lisp  ;; 起動時から有効にしておきたい補完コマンド用定義記述ファイル。
line-edit-init.lisp  ;; ユーザ用の初期設定項目を記述するためのファイル。
```

ただし、以下のファイルは起動時に読み込めないと一切の行編集機能が使えないので、これらのファイルだけは ~/.config/line-edit/ → カレント・ディレクトリ → ホームディレクトリ の順に探して読み込みます。このいずれにも設定がなかった場合はシステムとしてのデフォルト・エディタを設定します（line-edit-pkg.lisp 内のパラメータ *default-editor-mode* で定義しています。初期設定は emacs-mode）。ファイル内容を修正したときに混乱しないように ~/.config/line-edit だけに置くことをお勧めします。

```
emacs-mode.lisp
emacs-mode.help
vi-mode.lisp
vi-mode-function.lisp
vi-mode.help
WordMaster-mode-function.lisp
WordMaster-mode.help
WordMaster-mode.lisp
```

インストールは添付の Makefile で

「compile-」で始まるファイルは各パッケージをコンパイルする際に必要となる make コマンド用のファイルです。ソースコード群を置いているディレクトリと同じディレクトリ（~/Lisp/line-edit）に置きます。

現在、対応している Common Lisp 処理系は SBCL と CLISP の 2 種類です。開発は Ubuntu 24.04.4 LTS、CPU は 13th Gen Intel Core i7-13700×24 で行っています。その他の組み合わせでは検証していませんが、処理系依存機能、OS 依存機能についてはコンパイル時オプションで切り替えられるように配慮しています。

~/Lisp/line-edit ディレクトリに対して Common Lisp 処理系を実行できることを確認後、処理系が readline ライブラリを利用する場合は無効化してコンパイルします。SBCL は readline ライブラリを使用していますが CLISP はデフォルトで使用しているので無効化が必要です。SBCL と CLISP それぞれを以下のように alias 定義しておくとう便利です。

```
alias clisp='/usr/bin/clisp -ansi -q -q -disable-readline -i load-repl.lisp -repl -on-error abort'
alias sbcl='/usr/local/bin/sbcl --noinform --load load-repl.lisp'
```

なお、処理系が利用可能なメモリは多いほうが良いので仮に 64GB メモリのマシンであれば CLISP の場合は alias 定義末尾（行末のシングルスクォート「'」の手前）に `-m 65536MB`、SBCL の場合は `-dynamic-space-size 65536` と追加することをお勧めします。ここまでで必要な設定は完了です。

「Makefile」は依存関係によるコンパイル順序を制御する make コマンド用のデフォルト・ファイルです。

```
make lisp=sbcl
```

とすると SBCL を使用してコンパイルを行い

```
make lisp=clisp
```

とすると CLISP を使用してコンパイルを行います。

なお、

```
make lisp=sbcl clean (make lisp=clisp clean)
```

とするとコンパイルで生成されるオブジェクト・コード（.fasl、.fas）を削除できます。clean の代わりに check-clean とすると削除せず、削除対象となるファイル一覧の表示のみを行います。以上で、処理系起動後に

```
(load "load-repl.lisp")
```

とすれば line-edit-pkg システムが起動しますが、上述の alias 定義を行っていれば処理系実行コマンドだけで line-edit-pkg システムが起動します。無事にコンパイルできた場合、load-repl.lisp を自動ロードする alias 定義を行った状態であれば SBCL の場合、以下のような起動メッセージが表示されます。

```
[$2096] sbcl
=====
line-edit-pkg: Version 2026-05-16/SBCL
readline package for Common Lisp, by Common Lisp.
Copyright (C) Isao Daigo 2000-2026
Type (help) for help.
(select-language xx) for select message-language to xx.
xx = :ja, :en, :de, :fr, :zh-hans, :zh-hant, :ko, :zu
history-pkg commands available.
=====
Loading editor mode file... /home/daigo/.config/line-edit/emacs-mode.lisp
現在のエディタ・モードは emacs-mode です。
現時点で存在するパッケージ名に対する補完情報を作成しました。
[sbcl:(51.5MB) #1024]> █
```

CLISP の場合もほぼ同じですが、次のように処理系名部分が変わります。

```
[S2096] clisp
=====
line-edit-pkg: Version 2026-05-16/CLISP
readline package for Common Lisp, by Common Lisp.
Copyright (C) Isao Daigo 2000-2026
Type (help) for help.
(select-language xx) for select message-language to xx.
xx = :ja, :en, :de, :fr, :zh-hans, :zh-hant, :ko, :zu
history-pkg commands available.
=====
Loading editor mode file... /home/daigo/.config/line-edit/emacs-mode.lisp
現在のエディタ・モードは emacs-mode です。
現時点で存在するパッケージ名に対する補完情報を作成しました。
[clisp: #1024]>
```

以上でセットアップは終了です。

行編集コマンドは Emacs モード、vi モード、WordMaster¹モードを用意

行編集コマンドはコマンドを実行するためのキー列と実行用関数を紐づける仕組みを介しているので本物の Emacs 同様、柔軟に変更できます。line-edit-pkg 本体の編集用関数は Emacs モードの機能と 1 対 1 対応となるように作成しています。vi モードや WordMaster モードのコマンドで Emacs モードに存在しない機能は大抵の場合 Emacs モード用の関数を組み合わせる程度で実装できます。

```
Vi-mode-function.lisp
WordMaster-mode-function.lisp
```

は、それぞれに追加で必要となった関数定義を記述しているファイルです。vi-mode.lisp と WordMaster-mode.lisp から自動的に読み込まれます。emacs-mode.lisp ファイルの中身は global-set-key という関数によってキー列と、そのキー列をタイプしたときに呼び出される関数名を紐づける定義が羅列されています。一部を引用すると以下のようになっています。

```
(global-set-key "\\C-@"      #'set-mark-command)
(global-set-key "\\C- "     #'set-mark-command)
(global-set-key "\\C-^"     #'scroll-down)
(global-set-key "\\C-a"     #'beginning-of-line)
(global-set-key "\\C-b"     #'backward-char)
(global-set-key "\\C-d"     #'delete-char)
(global-set-key "[Delete]"  #'delete-char)
(global-set-key "\\C-e"     #'end-of-line)
(global-set-key "\\C-f"     #'forward-char)
(global-set-key "[Backspace]" #'delete-backward-char)
(global-set-key "[Tab]"     #'complete-symbol)
(global-set-key "[Enter]"   #'end-input)
(global-set-key "\\C-k"     #'kill-line)
```

¹ WordMaster というのは 1980 年代、8 ビット OS である CP/M 時代によく使われていたエディタです。資料が乏しいので、あまり作り込めていない可能性があります。当時のマニュアルを持っている方は、ぜひ作り込んで下さい。

1文字前方にカーソル（ポイント）を進める forward-char という関数は Control キーと f キーをタイプすると呼び出されるというように紐づけています。vi-mode.lisp、WordMaster-mode.lisp も同様です。
各エディタ・モードで利用できるコマンドは、各エディタ・モード時に

(help-edit)

とタイプすれば表示されます。エディタ・モードに応じて emacs-mode.help、vi-mode.help、WordMaster-mode.help というファイルを表示します。Emacs モードの場合は、

Command keys:

C-u [num]	Do next command 'num' times (optional, default=4 times).
C-f	Move forward one character.
M-f	Move forward one word.
M-C-d	Move forward down one level of parenthesis.
M-C-n	Move forward across one balanced group of parenthesis.
M-C-f	Move forward across one balanced symbolic-expression.
C-b	Move backward one character.
M-b	Move backward one word.
M-C-u	Move backward out of one level of parenthesis.
M-C-p	Move backward across one balanced group of parenthesis.
M-C-b	Move backward across one balanced symbolic-expression.
M-p	Move to balanced parenthesis.
M-n	Move to next word (like conventional editor).
C-a	Move to beginning of line.
C-e	Move to end of line.
M-a	Move to beginning of text.
M-e	Move to end of text.
C-p	Search history backward (previous history).
C-n	Search history forward (next history).
C-v	Scroll down one line.
M-v	Scroll up one line (or C-^).
C-x <	Scroll left.
C-x >	Scroll right.
M-<	Move to oldest history.
M->	Move to end of history (same as M-k).
C-i	Completion (or TAB).
C-s	Incremental search.
C-d	Delete character (or DEL).
M-d	Kill word.
M-C-k	Kill symbolic-expression.
C-k	Kill line from cursor to end.
M-k	Kill whole line and reset search position.
BS	Delete backward character (or C-h).
M-BS	Backward kill word (or M-C-h).
M-z <C>	Kill characters until <C> (zap-to-char).
M-\	Delete spaces and tabs around point.
M-s	Delete spaces and tabs around point, leaving one space.
C-t	Transpose characters.
M-t	Transpose words.
M-C-t	Transpose symbolic-expression.

M-(	Enclose following symbolic-expression in parenthesis.
C-@	Set region mark (or C-o)
M-@	Mark word (or M-o).
M-C-@	Mark symbolic-expression (or M-C-o).
C-x C-x	Exchange point and mark.
C-w	Kill-region (cut).
M-w	Kill-ring-save (copy).
M-C-w	Force next kills to append to previous kill.
C-y	Yank (paste from kill-ring).
M-y	Paste from kill-ring 2'nd last, 3'rd last,...
C-x r s<R>	Copy region to register <R> (<R> = any character).
C-x r i <R>	Insert text from register <R>.
C-x i	Insert file just before cursor position.
C-x C-f	Discard line then insert file.
C-x C-w	Write current line to specified file.
C-x C-s	Write current line to file.
M-u	Convert word to upper-case (upcase-word).
M-l	Convert word to lower-case (downcase-word).
M-c	Capitalize word.
C-l	Redraw line, delete control code.
C-g	Abort command.
C-x u	Undo previous changes (or C-_ or M-,).
C-x U	Undo previous undo (or M-.).
C-x (	Start keyboard macro.
C-u C-x (	Append to last macro defined.
C-x)	End Keyboard macro.
C-x e	Call last keyboard macro.
C-x q	Query user during kbd macro execution.
C-x C-k	Edit a keyboard macro.
C-x =	print information on cursor position.
C-q	Quote next character.
<ENTER>	end input.

unkeybinded-functions:

(last-kbd-macro)	returns list of last keyboard macro.
(kbd-macro-query-time n)	set how many sec. waiting for C-x q.
(kbd-macro-query-time)	returns how many sec. waiting for C-x q.
(save-macro fname)	saves kbd macro to fname.
(save-macro)	saves kbd macro to (macro-file).
(load-macro fname)	load kbd macro from fname.
(load-macro)	load kbd macro from (macro-file).
(macro-file fname)	set default load/save file for kbd macro.
(macro-file)	returns current load/save file for kbd macro.

というファイルが表示されます。C-x は Control キーを押しながら x キーをタイプすることを表し、M-x はメタ・キーを押しながら x キーをタイプすることを表します。ただし、大抵のキーボードにはメタ・キーがないので、代わりに alt キーを使います。また、ESC キーをタイプしてから x キーをタイプしても M-x と同じ意味になります。

できる限りオリジナルのエディタと同じ動作になるように定義しているので、詳しくは、それぞれのエディタのコマンド解説書を参照して下さい。

履歴機能は readline と C-shell の両スタイルをサポート

ひとつ上の行への移動など、行編集エディタでは意味のないコマンドは「ひとつ前の履歴への移動」機能と紐つけています。プロンプト行に何も入力していない状態で C-p (Emacs モード時) とタイプすれば、ひとつ前の行が表示され、そのままりターン・キーをタイプすれば表示されたコマンドが実行されます。

過去に入力したコマンドを効率よく見つけたい場合は、先頭の何文字かを入力してから C-p をタイプすると入力した文字と先頭部分が一致する履歴が表示されます。そのまま再度 C-p をタイプすると更に古い履歴のうち、先頭部分が一致する履歴が表示されます。これは保存されている履歴の先頭になるまで繰り返すことが出来ます。例えば過去に「(pushd :line-edit-pkg)」と入力したことがあり、この入力より前に「(pushd :support-functions)」と入力したことがある場合、

```
(pushd
```

と入力した状態で C-p をタイプすると、まず

```
(pushd :line-edit-pkg)
```

が表示され、そのまま更に C-p をタイプすると今度は

```
(pushd :support-functions)
```

と表示されます。表示された状態のままりターン・キーをタイプすると (pushd :support-functions) が実行されます。表示された履歴は行編集コマンドによって自由に修正できます。

履歴は初期状態では最大 1024 回分保管しています。このサイズは history-pkg.lisp 内の *hist-size* というパラメータで定義しています。メモリが許す限り、いくらでも大きい値を設定できます。保管個数の上限を超えた履歴情報は古い履歴から順に消えます。

REPL セッション終了時には自動的にすべての履歴がファイルに保存され、次回セッション開始時に保存された履歴が読み込まれます。

上の C-p による履歴検索は readline スタイルですが、C-shell スタイルでの履歴操作も行えます。この C-shell スタイルでの履歴操作は「!(感嘆符記号)」で始まるコマンドです。

!!

とタイプしてからリターン・キーをタイプすると直前に実行した行内容を再実行し、

!h

とタイプしてからリターン・キーをタイプすると直前に入力した最大 25 個分の履歴が履歴番号と共に表示されます。

```
[clisp:line-edit-pkg #1025]> !h
1000:(pushd :line-edit-pkg)
1001:(pushd :line-edit-pkg)
1002:(pushd :line-edit-pkg)
1003:(pushd :line-edit-pkg)
1004:(pushd :line-edit-pkg)
1005:(pushd :line-edit-pkg)
1006:(ls)
1007:(pushd :support-functions)
1008:(exchgd)
1009:(help 'ls)
1010:(help-f 'ls)
1011:(helpf 'ls)
1012:(select-language :en)
1013:(popd)
1014:(pushd :support-functions)
1015:(cd)
1016:(pushd :line-edit-pkg)
1017:(pushd :support-functions)
1018:(help-history)
1019:(help-complete)
1020:(select-language :ja)
1021:(help-complete)
1022:(cd)
1023:(pushd :line-edit-pkg)
1024:(pushd :line-edit-pkg)
T
[clisp:line-edit-pkg #1025]> █
```

再度実行したい入力の履歴番号が 1011 番であれば

!1011

と入力してからリターン・キーをタイプします。この C-shell スタイルの履歴操作では履歴保存用ファイルの内容を登録したエディタで直接編集することも出来ます。この設定は history-pkg.lisp 内のパラメータ *favorite-editor* で行っています。現在は “vi” が設定されています。

C-shell スタイルでの履歴機能は

!n	re-do n'th history. same as (do-history n).
!-n	re-do current minus n'th history. same as (do-history -n).
!!	re-do last input. same as !-1.

```

same as (do-history).
!do m n      do history from m to n.
              same as (do-history m n).
!history     print all history.
              same as (history).
!hist n      print last n history.
              same as (history n).
!h           print last histories.
              same as (last-histories).
!r or !read  read history from file.
              same as (read-history).
!w or !write write history to file.
              same as (write-history).
!c or !clear clear current history, not a file.
              same as (clear-history).
!edit        edit and re-read history file.
              same as (edit-history).
!ed n        edit and eval n'th history.
              same as (edit-history n).
!ed -n       edit and eval current minus n'th history.
              same as (edit-history -n).
!e           edit and eval last history.
              same as (edit-history -1).

un-keybinded-functions:
  (restore-history)  read history's history and history file.
  (reset-history)    reset history # to 0, then read history.
  (history-size)     returns current history buffer size.
  (history-size n)   set history buffer size to n.
  (last-histories)   returns current default number of prints.
  (last-histories n) set default number of prints. see !h.
  (history-with-number t) print history with number (default).
  (history-with-number nil) without number.
  (history-file fname) set history file to 'fname'.
  (history-file)      returns current history-file's name.
  (history-buffer fname) set history's buffer to 'fname'.
  (history-buffer)    returns current history buffer's name.

  (help-edit)        show current editor mode commands.

```

です。この一覧表は

```
(help-history)
```

とタイプすると表示できます。上記表内の「n」は履歴番号（整数）を表しています。

約 800 種類の標準関数・マクロなどの文法情報を登録済みの補完機能

補完機能は最初の何文字かを入力した状態でタブ・キー（初期設定）をタイプすると登録された候補の先頭部分と入力した文字が一致する候補を提示する機能です。確定キー（エディタ・モード毎に定義可能。Emacs モード時のデフォルト設定では C-j）をタイプすると確定します。提示された候補が求める候補でない場合は、そのままタブ・キーをタイプする毎に次々と別の候補を提示します。同じ行が書き換えられて新しい候補が表示されます。

```
[clisp:line-edit-pkg #1026]> (push [1/3] ([SPC]/NXT=C-n/PRV=C-p/OK=C-j/X=C-q))
```

候補は「短い候補」と「長い候補」から選べます。短い候補と長い候補はスペース・キーをタイプする毎に切り替わります。


```
[clisp:line-edit-pkg #1026]> (push item place) => new-place-value [1/3] ([SPC]/NXT=C-n/PRV=C-p/OK=C-j/X=C-q)
```

Common Lisp の約 800 種の標準関数の場合はスペース・キーで「長い候補」を選択すると引数と戻り値情報付きの文字列が表示されます。補完候補選択行に表示されているように C-j で確定。次の補完候補に移動するのであれば (タブ・キーに加えて) C-n、ひとつ前の補完候補に戻るのであれば C-p、補完候補の選択を中止するのであれば C-q をタイプします。この選択枝用のキーは、選択中のエディタ・モードに合わせて自由に変更できます。設定したキーは補完候補選択行に反映して表示されます。表示はされないものの、複数のキーが割り当てられている場合もあります。実際のキー割り当てがどうなっているかは各エディタ・モードの定義ファイル内の complete-symbol-set-key でのキー列と実行機能の割り当てで確認できます。

```
(complete-symbol-set-key "\\C-p"      :previous-candidate)    ;; ひとつ前の候補に戻る。
(complete-symbol-set-key "\\M-\\[A"    :previous-candidate)    ;; (↑) up arrow (HHKB=Fn + [).

(complete-symbol-set-key "\\C-n"      :next-candidate)         ;; 次の候補に移動する。
(complete-symbol-set-key "\\M-\\[B"    :next-candidate)         ;; (↓) down arrow (HHKB=Fn + /).
(complete-symbol-set-key "\\C-i"      :next-candidate)         ;; Tab.

(complete-symbol-set-key "\\C-j"      :current-candidate)      ;; 現在の候補に確定。
(complete-symbol-set-key "."          :current-candidate)      ;; [,][.][/]=[短][標準][長]
(complete-symbol-set-key "\\C-."      :current-candidate)      ;; [,][.][/]
(complete-symbol-set-key "\\M-."      :current-candidate)      ;; [,][.][/]

(complete-symbol-set-key "\\C-s"      :short-candidate)        ;; 短い候補に確定。
(complete-symbol-set-key ","          :short-candidate)        ;; [,][.][/]
(complete-symbol-set-key "\\C-,"      :short-candidate)        ;; [,][.][/]
(complete-symbol-set-key "\\M-,"      :short-candidate)        ;; [,][.][/]

(complete-symbol-set-key "\\C-l"      :long-candidate)         ;; 長い候補に確定。
(complete-symbol-set-key "/"          :long-candidate)         ;; [,][.][/]
(complete-symbol-set-key "\\C-/"      :long-candidate)         ;; [,][.][/]
(complete-symbol-set-key "\\M-/"      :long-candidate)         ;; [,][.][/]

(complete-symbol-set-key "r"          :redraw)                 ;; 現在の行を再描画し
(complete-symbol-set-key "\\C-r"      :redraw)                 ;; #\Newline 以外の制御文字を削除する。
(complete-symbol-set-key "\\M-r"      :redraw)

(complete-symbol-set-key "q"          :cancel)                 ;; 補完をキャンセル。
(complete-symbol-set-key "\\C-q"      :cancel)
(complete-symbol-set-key "\\M-q"      :cancel)

;; '(:previous-candidate :next-candidate :current-candidate :short-candidate :long-
candidate :redraw :cancel)
;;(setf *complete-symbol-used-key-def* '("C-p" "C-n" "." "," "/" "C-r" "q"))
(setf *complete-symbol-used-key-def* '("C-p" "C-n" "C-j" "C-s" "C-l" "C-r" "C-q"))
```

実際に補完候補選択枝として表示されるキーは最後の行の

```
(setf *complete-symbol-used-key-def* ...)
```

で定義されていますが、たとえば「ひとつ前の候補に戻る」ためのキー列は C-p の他に上矢印キーをタイプしたときに発生するキー列にも割り当てています。したがって上矢印キー (HHKB では Fn + [) でも「ひとつ前の候補に戻る」ことが出来ます²。

表示する補完候補が複数ある場合は、過去にその補完候補が選択された回数と最後に選択された日付・時刻を元に指数減衰曲線に沿った優先順位値を計算して優先順位が高い順に表示します。指数減衰曲線は

2 矢印キーやファンクション・キーなどの特殊キーをタイプしたときにキーコードを特殊なエスケープ・シーケンスとして返すことができるかどうかは端末次第です。一般的に DEC VT510 以上で拡張シーケンスのモード 1 を実装していれば対応しています。

Count	採用回数。
ΔT	現在時刻と最終参照時刻の差。
τ	半減期。

をパラメータとして

$$\text{Score} = \text{Count} \times 1/2^{(\Delta T/\tau)}$$

という式を計算して得られる曲線です。採用回数と現在時刻、および最終参照時刻の差は `line-edit-pkg` が自動的に設定します。半減期 (τ) は計算した優先順位を表す値が $1/2$ に減衰するまでの時間 (秒) です。

大きな値を設定すると順位は上がり難くなりますが、一旦上昇した順位は下がりにくくなります。逆に短い時間を設定すると順位は敏感に上下します。

安定した順位を保ちたい場合の半減期は2週間程度 (=14 日 × 24 時間 × 60 分 × 60 秒) が推奨値です。逆に順位が敏感に上下してほしい場合の半減期は3時間程度 (= 3 時間 × 60 分 × 60 秒) が推奨値です。この設定はサンプルとして付属している初期設定ファイル `line-edit-init.lisp` 内の

```
(line-edit-pkg:half-life-hour ...)
```

で設定しています。好みに合わせて書き換えて下さい。

補完候補に対する確定回数や確定した最終時刻情報は REPL セッション終了時に自動的に保存しています。したがって変動した補完候補の優先順位は次の REPL セッションに引き継がれます。

起動時に存在していたユーザ・パッケージに対しては、そのパッケージに移動 (=in-package) する毎に、外部シンボル、内部シンボル、継承シンボルを取得して補完候補として、その場で都度登録しています。ただし、これらの候補はいつ追加・削除されるか分からないので、REPL セッション中は優先順位は記録され変動しますが、REPL セッション終了時には情報は保存していません。

REPL プロンプトの定義方法

REPL プロンプトは直感的定義で自由に変更可能です。`history-pkg` 内の関数 `set-prompt-element` に表示したい要素を順に指定するだけです。指定できる要素は

:current-package	カレント・パッケージ名(ニックネームがあれば、ニックネームを返す)。
:original-package-name	カレント・パッケージ名を返す。ニックネームがあってもオリジナル名を返す。
:not-cl-user	cl-user パッケージでない場合のみパッケージ名を返す。
:date	今日の ISO フォーマットでの日付(YYYY-MM-DD)。
:time	24 時間フォーマットでの現在の時刻(HH:MM:DD)。
:time12	12 時間フォーマットでの現在の時刻(HH:MM{am,pm})。
:absolute-dir	カレント・ディレクトリ(絶対パス)。
:working-dir	カレント・ディレクトリ(相対パス)。
:working-dir-name	カレント・ディレクトリ名のみ。
:history-number	履歴番号。
:os-type	OS の種類。
:host-name	ホスト名/マシン名。
:machine-type	CPU タイプ。
:lisp-type	処理系名。
:lisp-version	処理系のバージョン番号。
:heap-size	使用しているヒープ領域のサイズ(MB)。sbcl のみ。
"string"	文字列。
:change-to-red	以降の表示を red に変更する。
:change-to-green	以降の表示を green に変更する。

:change-to-blue	以降の表示を blue に変更する。
:change-to-cyan	以降の表示を cyan に変更する。
:change-to-magenta	以降の表示を magenta に変更する。
:change-to-yellow	以降の表示を yellow に変更する。
:change-to-gray	以降の表示を gray に変更する。
:change-to-black	以降の表示を black に変更する。
#'func	関数 func の返す結果。

です。init-repl-prompt.lisp 内にプロンプトの設定情報を書いておくと起動時に指定したプロンプト情報が反映されます。

init-repl-prompt.lisp にはサンプルとして以下のような設定が書いてあります。

```
(package-util:set-package-name-case :downcase)           ;; パッケージ名を小文字で表示。
(history-pkg:set-prompt-attributes :color 'blue :bold t)  ;; 全体の初期色属性指定。
(history-pkg:set-prompt-element
  "["                ;; "["を表示。
  :lisp-type         ;; 処理系名。
  ":"               ;; ":"を表示。
  #'print-color-string:change-to-green                    ;; ここ以降は緑色。
  :not-cl-user       ;; カレント・パッケージが[:cl-user]でない場合のみ
                    ;; パッケージ名を表示。表示色は緑色。
  #'print-color-string:change-to-blue                      ;; ここ以降は再び青色。
  #+sbcl "("         ;; sbcl の場合は "(" を表示。
  #+sbcl #'print-color-string:change-to-red                ;; ここ以降は赤色。
  #+sbcl :heap-size   ;; sbcl の場合はヒープ・サイズを表示。
  #+sbcl #'print-color-string:change-to-blue                ;; ここ以降は青色。
  #+sbcl ")"         ;; sbcl の場合は ")" を表示。
  ;; " "            ;; 空白(" ")を表示。
  ;;:date           ;; ISO フォーマットでの日付(YYYY-MM-DD)
  ;; " "            ;; 空白(" ")を表示。
  ;;:time           ;; 時刻を 24 時間制(HH:MM:SS)で表示。
  " #"
  :history-number "]"> "      ;; 文字の色や属性指定はここでリセットされる。
) ;; end history-pkg:set-prompt-element
```

この設定で SBCL の場合は起動時に次のように表示されます。

```
[$2097] sbcl
=====
line-edit-pkg: Version 2026-05-16/SBCL
readline package for Common Lisp, by Common Lisp.
Copyright (C) Isao Daigo 2000-2026
Type (help) for help.
(select-language xx) for select message-language to xx.
xx = :ja, :en, :de, :fr, :zh-hans, :zh-hant, :ko, :zu
history-pkg commands available.
=====
Loading editor mode file... /home/daigo/.config/line-edit/emacs-mode.lisp
現在のエディタ・モードは emacs-mode です。
現時点で存在するパッケージ名に対する補完情報を作成しました。
[sbcl:(51.6MB) #1024]> |
```

全体の色属性が blue, bold と設定されているので青字で "[" が表示され、次に処理系名 (:lisp-type) が "sbcl" と表示され、文字列 ":"、起動時は Common-lisp-User パッケージなのでパッケージ名表示なし、処理系が SBCL の場合は文字列 '(' を表示し、以降の表示色を赤に変更、処理系が SBCL の場合はヒープ・サイ

ズを表示 (SBCL 独自機能)、以降の表示色を再び青に変更、処理系が SBCL の場合、文字列")"を表示、文字列" #"を表示してから履歴番号を表示して、最後に文字列"]> "を表示して終了。

以上を REPL プロンプトが更新されるごとに実行します。したがって `:time` というキーワードが書いてあれば、毎回最新の時分秒が表示されます。

もし、用意してあるキーワードでは不足する場合は `#'func` という形式で関数を指定できるので、好みの結果を返す関数を用意することで更に自由にカスタマイズ出来ます。

パッケージ移動は C-shell のディレクトリ移動スタイルでスタック管理

パッケージ単位で大規模プログラムを管理できるのは Common Lisp の大きな特徴のひとつです。しかし、複数のパッケージをロードした状態では、自分が今どのパッケージにいるのかを視覚的に知る手段がありません。本プログラムは REPL プロンプトに現在ユーザがいるパッケージ名を表示でき、パッケージ間を移動する履歴をスタックで管理できます。

<code>(pushd :package-name)</code>	現在のパッケージから <code>:package-name</code> パッケージに移動。
<code>(popd)</code>	現在のパッケージからひとつ前にいたパッケージに移動。
<code>(rotdd)</code>	ディレクトリ・スタックを下方方向に回転する。
<code>(rotdu)</code>	ディレクトリ・スタックを上方方向に回転する。
<code>(exchgd)</code>	ディレクトリ・スタックの上ふたつの順序を入れ替える。
<code>(dirs)</code>	ディレクトリ・スタックの状態を表示する。
<code>(ls)</code>	現在のパッケージ内の外部・内部・継承シンボルを表示する。

`(auto-show-dirs t)` という設定があるとパッケージ移動毎に自動的に `(dirs)` が実行されます。関数 `ls` はいくつかのキーワード・パラメータがあり、キーワード・パラメータで指定された内容だけを表示することが出来ます。

<code>:string "文字列"</code>	"文字列"を含むシンボルのみを表示する。デフォルトは <code>"" (=all)</code> 。
<code>:package 'パッケージ'</code>	パッケージ文字列かパッケージ・オブジェクト。デフォルトは <code>*package*</code> 。
<code>:external [t/nil]</code>	外部名シンボルを表示する。default <code>[t]</code> 。
<code>:internal [t/nil]</code>	内部名シンボルを表示する。default <code>[nil]</code> 。
<code>:inherited [t/nil]</code>	継承シンボルを表示する。default <code>[nil]</code> 。
<code>:verbose [t/nil]</code>	表示シンボルの属性なども表示する。default <code>[t]</code> 。
<code>:quiet [t/nil]</code>	属性ごとのタイトル表示を行わない。default <code>[nil]</code> 。

たとえば

```
(ls :string "push" :external t)
```

として実行すると文字列 `push` で始まる外部シンボルのみを表示します。

```
[sbcl:line-edit-pkg(53.9MB) #1025]> (ls :string "push" :external t)
External symbols in line-edit-pkg:
  (Function) push-command-trace-list
  (Function) push-keyboard-macro
  (Function) push-kill-ring
  (Function) push-redo-info
  (Function) push-undo-info
5 symbols
Total: 5 symbols
[sbcl:line-edit-pkg(55.6MB) #1026]> █
```

外部シンボル名の前にはシンボルの種別（関数、定数など）も表示されます。:verbose キーワードに nil を与えて実行すると表示されません。

```
[sbcl:line-edit-pkg(56.5MB) #1026]> (ls :string "push" :external t :verbose nil)
External symbols in line-edit-pkg:
  push-command-trace-list
  push-keyboard-macro
  push-kill-ring
  push-redo-info
  push-undo-info
[sbcl:line-edit-pkg(58.8MB) #1027]> █
```

パッケージ間の移動は前述の各スタック・スタイルでの移動に加えて従来どおりの

(in-package :package-name)

スタイルでも行えます。このスタイルでカレント・パッケージを変更しても移動したことを検知してプロンプトにパッケージ名を表示している場合はパッケージ名が切り替わります。動作としては

(cd :package-name)

と同じことになります。

プログラムからの主要メッセージは多言語対応

プログラムからの主要メッセージはAI による事前翻訳で用意した言語での表示に自由に切り替え可能です。デフォルトでは日本語メッセージですが、現在、日本語 (:ja) の他に

- 英語 (:en)
- ドイツ語 (:de)
- フランス語 (:fr)
- 簡体中国語 (:zh-hans)
- 繁体中国語 (:zh-hant)

- 韓国語 (:ko)
- ズールー語 (:zu)

のメッセージを用意してあります。切り替えは

```
(select-language :en)
```

で英語に切り替わります。これらの設定はシステム全体の初期化ファイル「line-edit-init.lisp」内に記述しておけば起動時には設定が完了しています。またREPLセッション途中での切り替えも出来ます。

Ubuntuは南アフリカ出身のMark Shuttleworth氏が創設し、「他者への思いやり」、「人間性」などの意味を持つ言葉であることへの敬意から南アフリカ周辺で話されているズールー語にも対応しています。「人は人によって人になる」という哲学が背景にあるそうです。

以上。

mail2daigo@gmail.com